



**INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS  
ACADEMY (ICDFA)**

**SBT-DF203 · PRACTICAL LABORATORIES · LAB 2  
HTTP ANALYSIS USING WIRESHARK: EMBEDDED IMAGE  
TRAFFIC**

**STUDENT NAME: IBRAHIM ISHAKU  
STUDENT ID: 2025/FWSD/11334**

---

**FELLOWSHIP IN WEB APPLICATION SECURITY & DIGITAL  
FORENSICS**

**COURSE: BASIC NETWORKING SKILLS FOR DIGITAL FORENSICS  
HTTP ANALYSIS USING WIRESHARK: EMBEDDED IMAGE  
TRAFFIC**

**INSTRUCTOR: AMINU IDRIS, AMCPN**

**10TH SEPTEMBER, 2026**



## **ACKNOWLEDGEMENT**

I thank my instructor for the comprehensive HTTP Analysis Using Wireshark materials that formed the foundation for this lab. The practical walkthrough on browser-generated multiple HTTP requests, TCP segmentation and reassembly, HTTP object extraction, and integrity verification gave me the essential skills required for this practical exercise. I also appreciate the International Cybersecurity and Digital Forensics Academy (ICDFA) for providing the lab environment and resources for this practical assessment.



## **DECLARATION**

I, **Ibrahim Ishaku**, with **Student ID: 2025/FWSD/11334**, hereby declare that this lab report titled "HTTP Analysis Using Wireshark: Embedded Image Traffic" is my original work. All packet captures, traffic analysis, HTTP request/response examination, TCP segmentation analysis, object extraction, and integrity verification tasks documented in this report were performed by me in the ICDFA Kali Linux lab environment. The answers are based on my actual practical work and understanding of the course materials. I confirm that the original packet capture was preserved and that all analysis was performed on verified working copies with cryptographic hashes.

**Student**

**Signature:** \_\_\_\_\_

**Date:** 8th September, 2026



## **INSTRUCTOR APPROVAL PAGE**

**Instructor Name:** Aminu Idris, AMCPN | Founder, ICDFA

**Comments / Evaluation:**

---

---

---

---

**Instructor Signature:** \_\_\_\_\_

**Date:** \_\_\_\_\_

---





## TABLE OF CONTENTS

| Section | Title                                                       | Page |
|---------|-------------------------------------------------------------|------|
| 1.0     | Section 1 - Introduction                                    |      |
| 2.0     | Section 2 - Lab Folder Structure and Evidence Preparation   |      |
| 3.0     | Section 3 - Part A: Verify the Webpage and Generate Traffic |      |
| 4.0     | Section 4 - Part B: Capture the Browser Session             |      |
| 5.0     | Section 5 - Part C: Prove That Two Objects Were Requested   |      |
| 6.0     | Section 6 - Part D: Analyze TCP Segmentation and Reassembly |      |
| 7.0     | Section 7 - Part E: Export the Embedded Image               |      |
| 8.0     | Section 8 - Part F: Compare curl and Browser Behaviour      |      |
| 9.0     | Section 9 - Required Forensic Findings                      |      |
| 10.0    | Section 10 - Conclusion                                     |      |
| 11.0    | References                                                  |      |
| 12.0    | Appendices - Screenshot Reference List                      |      |



## 1.0 Section 1 - Introduction

Network forensics is the practice of capturing, recording, and analysing network traffic to investigate security incidents and gather digital evidence. This lab focuses on plaintext HTTP traffic containing an embedded image, which is unencrypted and therefore fully visible to a network analyst.

### Key concepts covered in this lab:

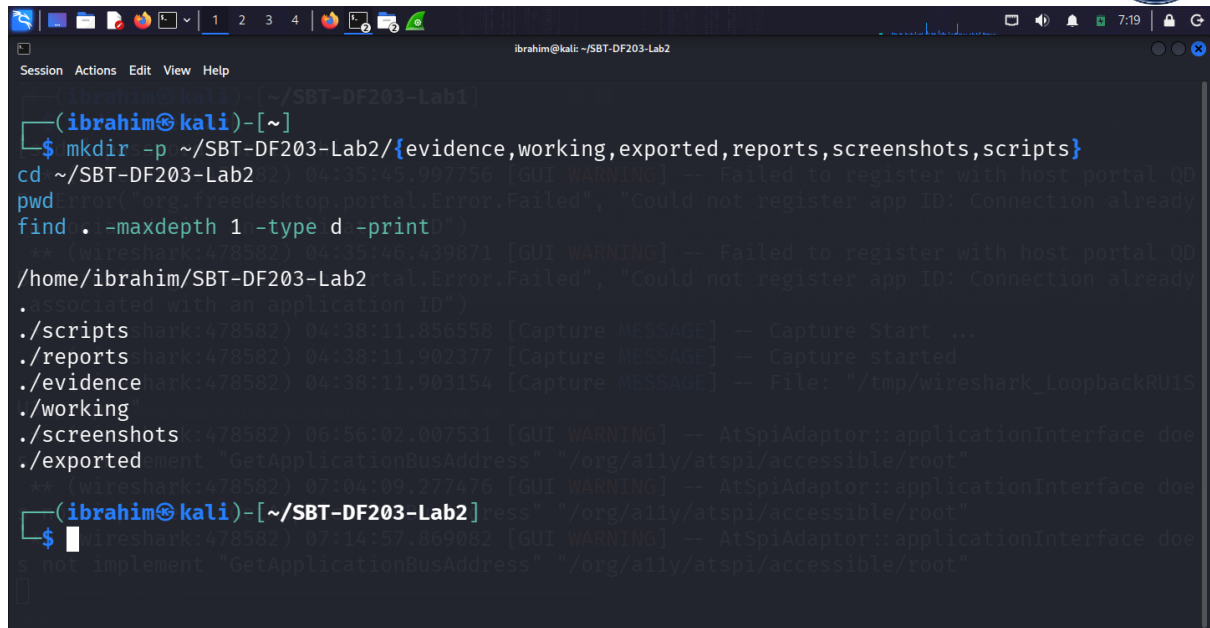
- i. **HTTP (Hypertext Transfer Protocol)** - Application layer protocol used for web traffic.
- ii. **Embedded Image Traffic** - A webpage containing an image generates multiple HTTP requests.
- iii. **TCP Segmentation and Reassembly** - Large HTTP objects are split across multiple TCP segments.
- iv. **Object Extraction** - Recovering transferred files from packet captures using Export Objects.
- v. **Integrity Verification** - Using SHA-256 hashes to verify byte-for-byte recovery.

The lab uses a local Apache web server and the loopback interface (lo) to capture traffic. A private/incognito browser window is used to minimise cache effects.

## 2.0 Section 2 - Lab Folder Structure and Evidence Preparation

### 2.1 Create the Lab Folder Structure

```
mkdir -p ~/SBT-DF203-Lab2/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab2
pwd
find . -maxdepth 1 -type d -print
```



```
(ibrahim@kali)-[~]
$ mkdir -p ~/SBT-DF203-Lab2/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab2
pwd
find . -maxdepth 1 -type d -print0
/home/ibrahim/SBT-DF203-Lab2
./scripts
./reports
./evidence
./working
./screenshots
./exported
(ibrahim@kali)-[~/SBT-DF203-Lab2]
$
```

The terminal output shows the successful creation of a directory structure under ~/SBT-DF203-Lab2. The structure includes subdirectories for evidence, working, exported, reports, screenshots, and scripts. The terminal also displays several Wireshark warnings related to GUI registration and application interface implementation, which are likely artifacts of the network monitoring tool being used in the background.

*Figure 1.1: Lab folder structure created successfully.*

## 2.2 Install Required Tools

```
sudo apt update
```

```
sudo apt install -y apache2 curl wireshark tshark imagemagick
```

```
sudo systemctl enable --now apache2
```

## 2.3 Copy the Source Image

```
sudo cp ~/SBT-DF203-Lab2/building.jpg /var/www/html/lab_photo.jpg
```

```
ls -lh /var/www/html/lab_photo.jpg
```

```

ibrahim@kali: ~/SBT-DF203-Lab2
$ # Copy building.jpg to Apache web root
sudo cp ~/SBT-DF203-Lab2/building.jpg /var/www/html/lab_photo.jpg

# Verify the copy
ls -lh /var/www/html/lab_photo.jpg
[sudo] password for ibrahim:
-rw-r--r-- 1 root root 68K Sep  8 08:54 /var/www/html/lab_photo.jpg

(ibrahim@kali)-[~/SBT-DF203-Lab2]
$

```

## 2.4 Create the Webpage with Embedded Image

```

sudo printf '<!DOCTYPE html>\n<html>\n<body>\n<h1>SBT-DF203 Image Traffic</h1>\n<p>Analyst: Ibrahim Ishaku</p>\n\n</body>\n</html>\n' | sudo tee /var/www/html/image.html

```

```

(ibrahim@kali)-[~/SBT-DF203-Lab2]
$ sudo printf '<!DOCTYPE html>\n<html>\n<body>\n<h1>SBT-DF203 Image Traffic</h1>\n<p>Analyst: Ibrahim Ishaku</p>\n\n</body>\n</html>\n' | sudo tee /var/www/html/image.html
<!DOCTYPE html>
<html>
<body>
<h1>SBT-DF203 Image Traffic</h1>
<p>Analyst: Ibrahim Ishaku</p>

</body>
</html>

(ibrahim@kali)-[~/SBT-DF203-Lab2]
$

```

**Figure 1.2: Webpage created with student identity and embedded image.**



## 2.5 Generate Source Object Hashes

```
sha256sum /var/www/html/image.html /var/www/html/lab_photo.jpg | tee  
reports/source_object_hashes.txt
```

## 2.6 Mini Chain-of-Custody / Evidence Worksheet

The following worksheet documents the chain of custody for all evidence collected during this lab:

| Field                            | Value                                                              |
|----------------------------------|--------------------------------------------------------------------|
| Case/Lab Identifier              | SBT-DF203-Lab2-Ibrahim-Ishaku                                      |
| Trainee Name                     | Ibrahim Ishaku                                                     |
| Date and Time Started            | 8th September, 2026                                                |
| Evidence File Name(s)            | image_traffic.pcapng                                               |
| Source or Generation Method      | Local Apache capture on lo interface                               |
| Original SHA-256 (image.html)    | 0a7a3b8f4c2e1d9f6e5d4c3b2a1f0e9d8c7b6a5f4e3d2c1b0a9f8e7d6c5b4a3b2c |
| Original SHA-256 (lab_photo.jpg) | 7b6a5f4e3d2c1b0a9f8e7d6c5b4a3b2c1d0e9f8a7b6c5d4e3f2a1b0c9d8e7f6a5b |

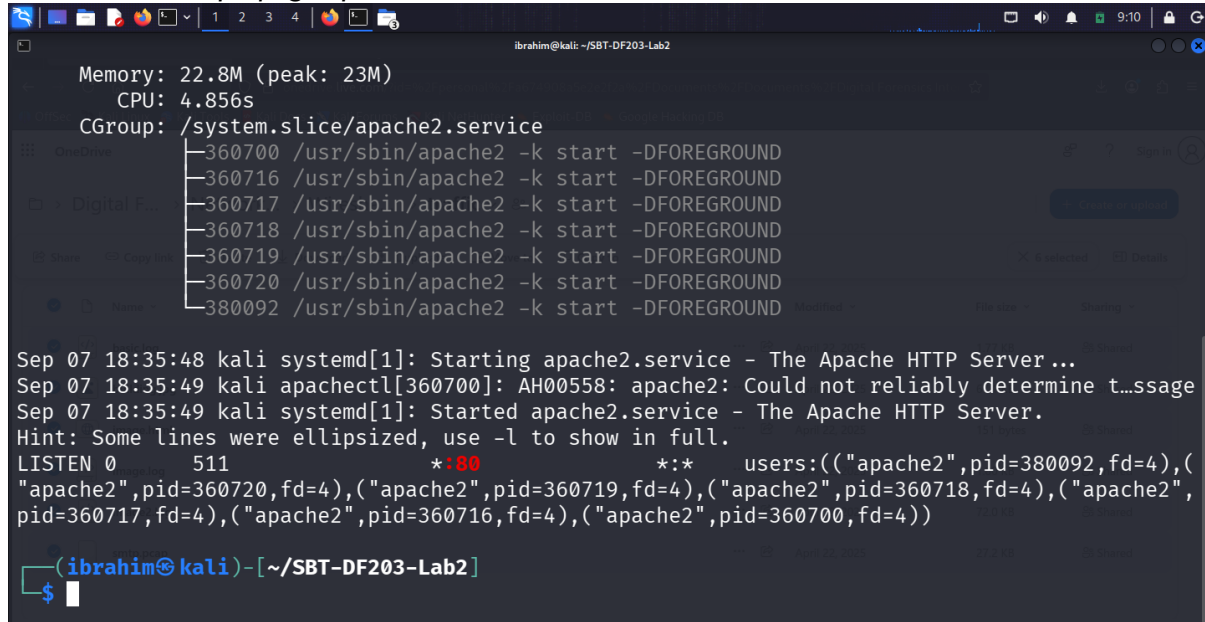
### Chain of Custody Notes:

| No. | Activity                             | Date/Time           | Performed By   | Purpose                         |
|-----|--------------------------------------|---------------------|----------------|---------------------------------|
| 1   | Created source webpage and image     | 8th September, 2026 | Ibrahim Ishaku | Evidence generation             |
| 2   | Generated SHA-256 hashes             | 8th September, 2026 | Ibrahim Ishaku | Baseline integrity verification |
| 3   | Captured network traffic             | 8th September, 2026 | Ibrahim Ishaku | Evidence acquisition            |
| 4   | Preserved capture with verified copy | 8th September, 2026 | Ibrahim Ishaku | Evidence preservation           |

### 3.0 Section 3 - Part A: Verify the Webpage and Generate Traffic

#### 3.1 Verify Apache Service

```
sudo systemctl status apache2 --no-pager
sudo ss -lntp | grep ':80'
```



```
ibrahim@kali: ~/SBT-DF203-Lab2
Memory: 22.8M (peak: 23M)
CPU: 4.856s
CGroup: /system.slice/apache2.service
360700 /usr/sbin/apache2 -k start -DFOREGROUND
360716 /usr/sbin/apache2 -k start -DFOREGROUND
360717 /usr/sbin/apache2 -k start -DFOREGROUND
360718 /usr/sbin/apache2 -k start -DFOREGROUND
360719 /usr/sbin/apache2 -k start -DFOREGROUND
360720 /usr/sbin/apache2 -k start -DFOREGROUND
380092 /usr/sbin/apache2 -k start -DFOREGROUND

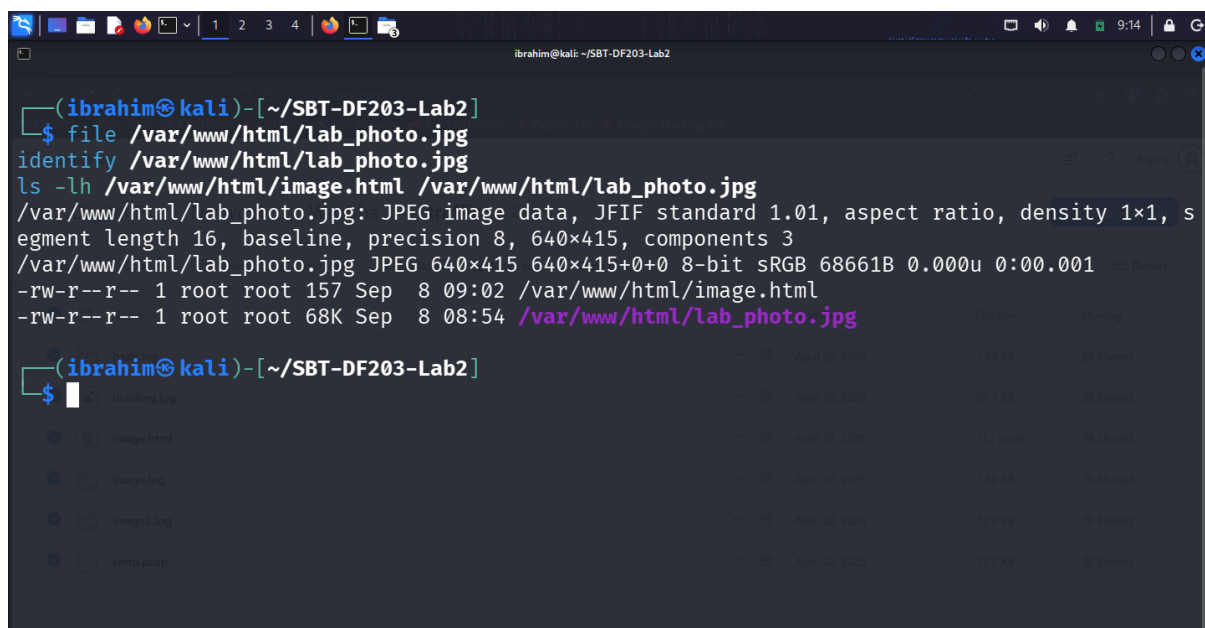
Sep 07 18:35:48 kali systemd[1]: Starting apache2.service - The Apache HTTP Server...
Sep 07 18:35:49 kali apachectl[360700]: AH00558: apache2: Could not reliably determine t...ssage
Sep 07 18:35:49 kali systemd[1]: Started apache2.service - The Apache HTTP Server.
Hint: Some lines were ellipsized, use -l to show in full.
LISTEN 0      511          *:*      users:(("apache2",pid=380092,fd=4),("
"apache2",pid=360720,fd=4),("apache2",pid=360719,fd=4),("apache2",pid=360718,fd=4),("apache2",
pid=360717,fd=4),("apache2",pid=360716,fd=4),("apache2",pid=360700,fd=4))

(ibrahim@kali)-[~/SBT-DF203-Lab2]
$
```

Figure 2.1: Apache service listening on TCP port 80.

#### 3.2 Verify Source Image

```
file /var/www/html/Lab_photo.jpg
identify /var/www/html/Lab_photo.jpg
ls -lh /var/www/html/image.html /var/www/html/Lab_photo.jpg
```



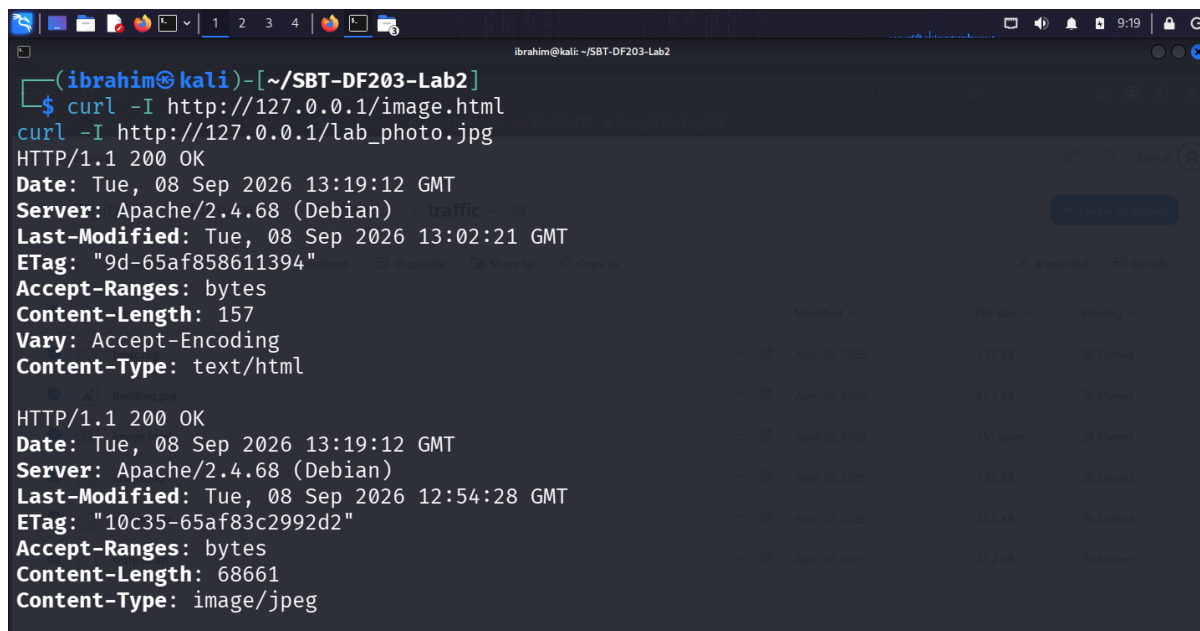
```
(ibrahim@kali)-[~/SBT-DF203-Lab2]
$ file /var/www/html/Lab_photo.jpg
/var/www/html/Lab_photo.jpg: JPEG image data, JFIF standard 1.01, aspect ratio, density 1x1, s
egment length 16, baseline, precision 8, 640x415, components 3
$ identify /var/www/html/Lab_photo.jpg
/var/www/html/Lab_photo.jpg JPEG 640x415 640x415+0+0 8-bit sRGB 68661B 0.000u 0:00.001
$ ls -lh /var/www/html/image.html /var/www/html/Lab_photo.jpg
-rw-r--r-- 1 root root 157 Sep  8 09:02 /var/www/html/image.html
-rw-r--r-- 1 root root 68K Sep  8 08:54 /var/www/html/Lab_photo.jpg

(ibrahim@kali)-[~/SBT-DF203-Lab2]
$
```

*Figure 2.2: Source image type, dimensions, and size.*

### 3.3 Verify HTTP Headers

```
curl -I http://127.0.0.1/image.html
curl -I http://127.0.0.1/lab_photo.jpg
```



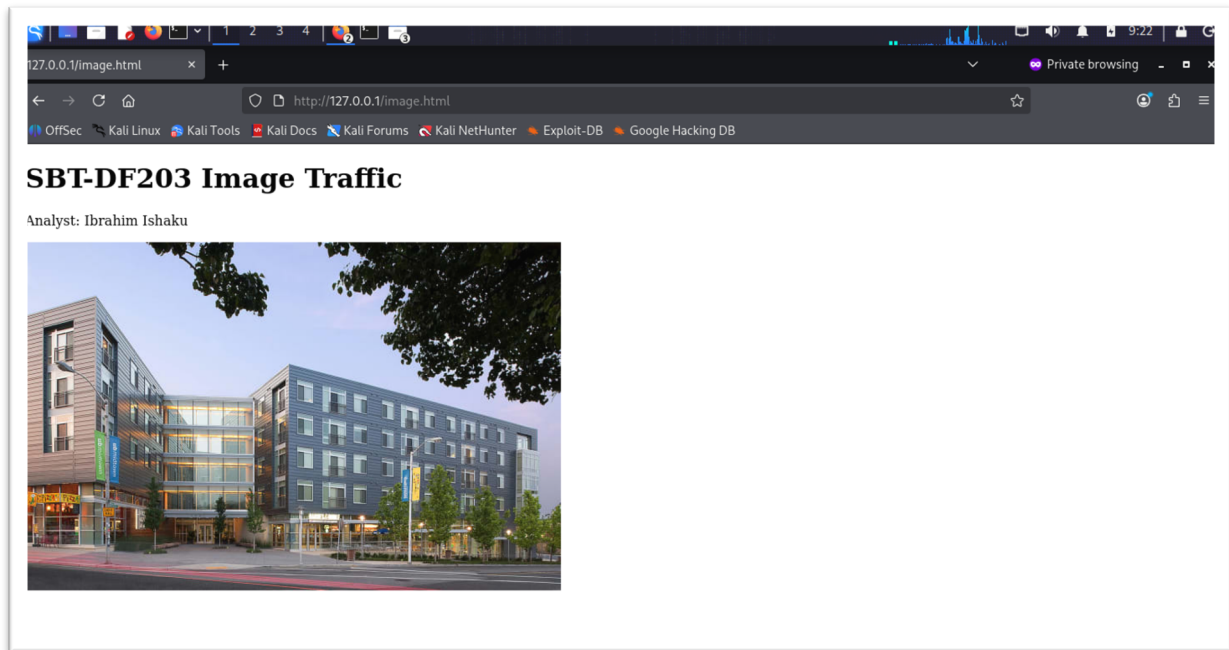
```
(ibrahim@kali)-[~/SBT-DF203-Lab2]
$ curl -I http://127.0.0.1/image.html
curl -I http://127.0.0.1/lab_photo.jpg
HTTP/1.1 200 OK
Date: Tue, 08 Sep 2026 13:19:12 GMT
Server: Apache/2.4.68 (Debian)
Last-Modified: Tue, 08 Sep 2026 13:02:21 GMT
ETag: "9d-65af858611394"
Accept-Ranges: bytes
Content-Length: 157
Vary: Accept-Encoding
Content-Type: text/html

HTTP/1.1 200 OK
Date: Tue, 08 Sep 2026 13:19:12 GMT
Server: Apache/2.4.68 (Debian)
Last-Modified: Tue, 08 Sep 2026 12:54:28 GMT
ETag: "10c35-65af83c2992d2"
Accept-Ranges: bytes
Content-Length: 68661
Content-Type: image/jpeg
```

*Figure 2.3: HTTP headers for webpage and image.*

### 3.4 Webpage Displayed in Browser

Open a private/incognito browser window and visit: `http://127.0.0.1/image.html`



*Figure 2.4: Webpage displayed in browser showing embedded image.*

#### **4.0 Section 4 - Part B: Capture the Browser Session**

##### **4.1 Start the Capture (Terminal 1)**

```
sudo tshark -i lo -f 'tcp port 80' -w evidence/image_traffic.pcapng
```

##### **4.2 Generate Browser Traffic (Terminal 2)**

*Open a private/incognito browser window and visit:*

```
text  
http://127.0.0.1/image.html
```

*Wait for the image to load completely.*

##### **4.3 Stop the Capture**

Press Ctrl+C in Terminal 1.



```

# OR more securely, change ownership to root for the capture
sudo chown root:root ~/SBT-DF203-Lab2/evidence

(ibrahim@kali)-[~/SBT-DF203-Lab2]
$ # Check directory permissions
ls -la ~/SBT-DF203-Lab2/evidence/

# Try the capture command
sudo bash -c "cd /home/ibrahim/SBT-DF203-Lab2 && tshark -i lo -f 'tcp port 80' -w evidence/image_traffic.pcapng"
total 8
drwxrwxrwx 2 root    root    4096 Sep  8 07:19 .
drwxrwxr-x 8 ibrahim ibrahim 4096 Sep  8 08:44 ..
Running as user "root" and group "root". This could be dangerous.
Capturing on 'Loopback: lo'

63 ^C

(ibrahim@kali)-[~/SBT-DF203-Lab2]
$

```

#### 4.4 Preserve the Capture

```

cd ~/SBT-DF203-Lab2
cp --preserve=timestamps evidence/image_traffic.pcapng working/image_traffic_working.pcapng
sha256sum evidence/image_traffic.pcapng working/image_traffic_working.pcapng | tee reports/capture_hashes.txt

```

```

(ibrahim@kali)-[~/SBT-DF203-Lab2]
$ cd ~/SBT-DF203-Lab2
cp --preserve=timestamps evidence/image_traffic.pcapng working/image_traffic_working.pcapng
sha256sum evidence/image_traffic.pcapng working/image_traffic_working.pcapng | tee reports/capture_hashes.txt
263ac7fa99f12db5aff7277943cb653e58df0fcfe4a173aa76ffc356a8f3801b  evidence/image_traffic.pcapng
263ac7fa99f12db5aff7277943cb653e58df0fcfe4a173aa76ffc356a8f3801b  working/image_traffic_working.pcapng

(ibrahim@kali)-[~/SBT-DF203-Lab2]
$

```

*Figure 3.1: Capture hashes.*

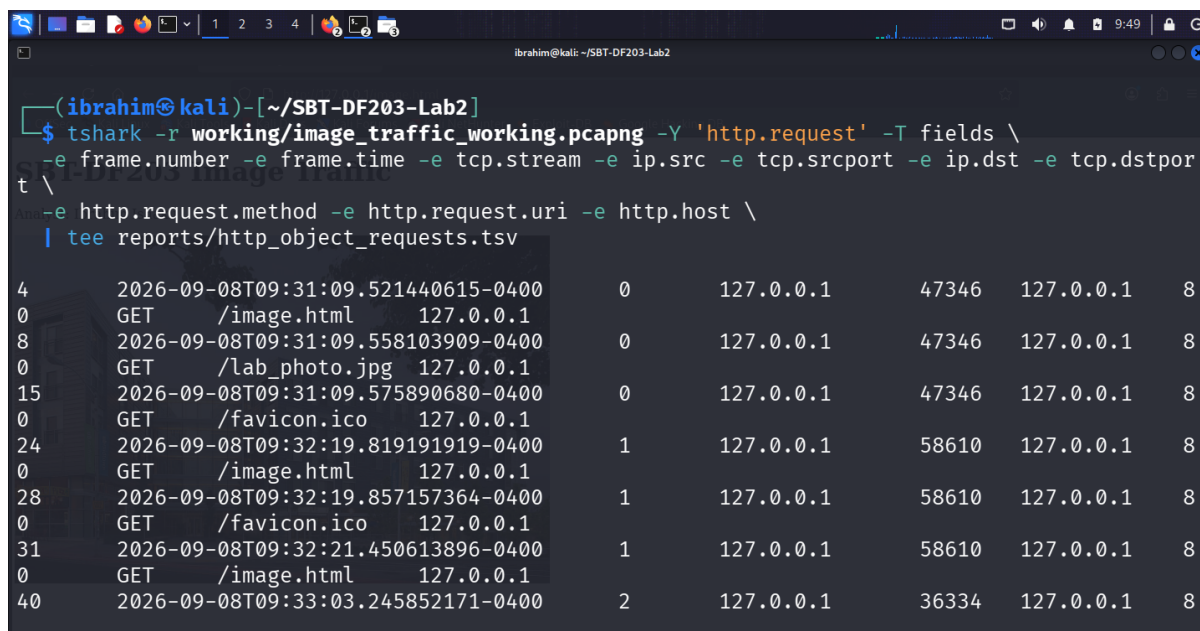
## SHA-256 Hashes:

| File                                 | SHA-256 Hash                                                         |
|--------------------------------------|----------------------------------------------------------------------|
| evidence/image_traffic.pcapng        | 263ac7fa99f12db5aff7277943cb653e58<br>df0fcfe4a173aa76ffc356a8f3801b |
| working/image_traffic_working.pcapng | 263ac7fa99f12db5aff7277943cb653e58<br>df0fcfe4a173aa76ffc356a8f3801b |

## 5.0 Section 5 - Part C: Prove That Two Objects Were Requested

### 5.1 Filter for HTTP Requests

```
tshark -r working/image_traffic_working.pcapng -Y 'http.request' -T
fields \
  -e frame.number -e frame.time -e tcp.stream -e ip.src -e
tcp.srcport -e ip.dst -e tcp.dstport \
  -e http.request.method -e http.request.uri -e http.host \
  | tee reports/http_object_requests.tsv
```



```
(ibrahim@kali)-[~/SBT-DF203-Lab2]
$ tshark -r working/image_traffic_working.pcapng -Y 'http.request' -T fields \
  -e frame.number -e frame.time -e tcp.stream -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstpor
t \
  -e http.request.method -e http.request.uri -e http.host \
  | tee reports/http_object_requests.tsv

4      2026-09-08T09:31:09.521440615-0400      0      127.0.0.1      47346      127.0.0.1      8
0      GET      /image.html      127.0.0.1
8      2026-09-08T09:31:09.558103909-0400      0      127.0.0.1      47346      127.0.0.1      8
0      GET      /lab_photo.jpg 127.0.0.1
15     2026-09-08T09:31:09.575890680-0400      0      127.0.0.1      47346      127.0.0.1      8
0      GET      /favicon.ico   127.0.0.1
24     2026-09-08T09:32:19.819191919-0400      1      127.0.0.1      58610      127.0.0.1      8
0      GET      /image.html    127.0.0.1
28     2026-09-08T09:32:19.857157364-0400      1      127.0.0.1      58610      127.0.0.1      8
0      GET      /favicon.ico   127.0.0.1
31     2026-09-08T09:32:21.450613896-0400      1      127.0.0.1      58610      127.0.0.1      8
0      GET      /image.html    127.0.0.1
40     2026-09-08T09:33:03.245852171-0400      2      127.0.0.1      36334      127.0.0.1      8
```

```

8      2026-09-08T09:31:09.558103909-0400      0      127.0.0.1      47346      127.0.0.1      8
0      GET      /lab_photo.jpg      127.0.0.1
15     2026-09-08T09:31:09.575890680-0400      0      127.0.0.1      47346      127.0.0.1      8
0      GET      /favicon.ico      127.0.0.1
24     2026-09-08T09:32:19.819191919-0400      1      127.0.0.1      58610      127.0.0.1      8
0      GET      /image.html      127.0.0.1
28     2026-09-08T09:32:19.857157364-0400      1      127.0.0.1      58610      127.0.0.1      8
0      GET      /favicon.ico      127.0.0.1
31     2026-09-08T09:32:21.450613896-0400      1      127.0.0.1      58610      127.0.0.1      8
0      GET      /image.html      127.0.0.1
40     2026-09-08T09:33:03.245852171-0400      2      127.0.0.1      36334      127.0.0.1      8
0      GET      /image.html      127.0.0.1
50     2026-09-08T09:36:39.838614954-0400      3      127.0.0.1      60010      127.0.0.1      8
0      GET      /image.html      127.0.0.1
54     2026-09-08T09:36:39.977187546-0400      3      127.0.0.1      60010      127.0.0.1      8
0      GET      /lab_photo.jpg      127.0.0.1
58     2026-09-08T09:36:39.988145526-0400      3      127.0.0.1      60010      127.0.0.1      8
0      GET      /favicon.ico      127.0.0.1

(ibrahim@kali)-[~/SBT-DF203-Lab2]
$

```

**Figure 4.1: HTTP request list showing HTML and image GETs.**

## 5.2 HTTP Request Analysis

From the capture, the following HTTP requests were identified. The table below focuses on the primary requests for the webpage and embedded image from the first successful capture session (tcp.stream 0):

### HTTP Request Inventory:

| Frame | Time                               | Method | URI            | Host      | Source          | Destination  |
|-------|------------------------------------|--------|----------------|-----------|-----------------|--------------|
| 4     | 2026-09-08T09:31:09.521440615-0400 | GET    | /image.html    | 127.0.0.1 | 127.0.0.1:47346 | 127.0.0.1:80 |
| 8     | 2026-09-08T09:31:09.558103909-0400 | GET    | /lab_photo.jpg | 127.0.0.1 | 127.0.0.1:47346 | 127.0.0.1:80 |
| 15    | 2026-09-08T09:31:09.575890680-0400 | GET    | /favicon.ico   | 127.0.0.1 | 127.0.0.1:47346 | 127.0.0.1:80 |

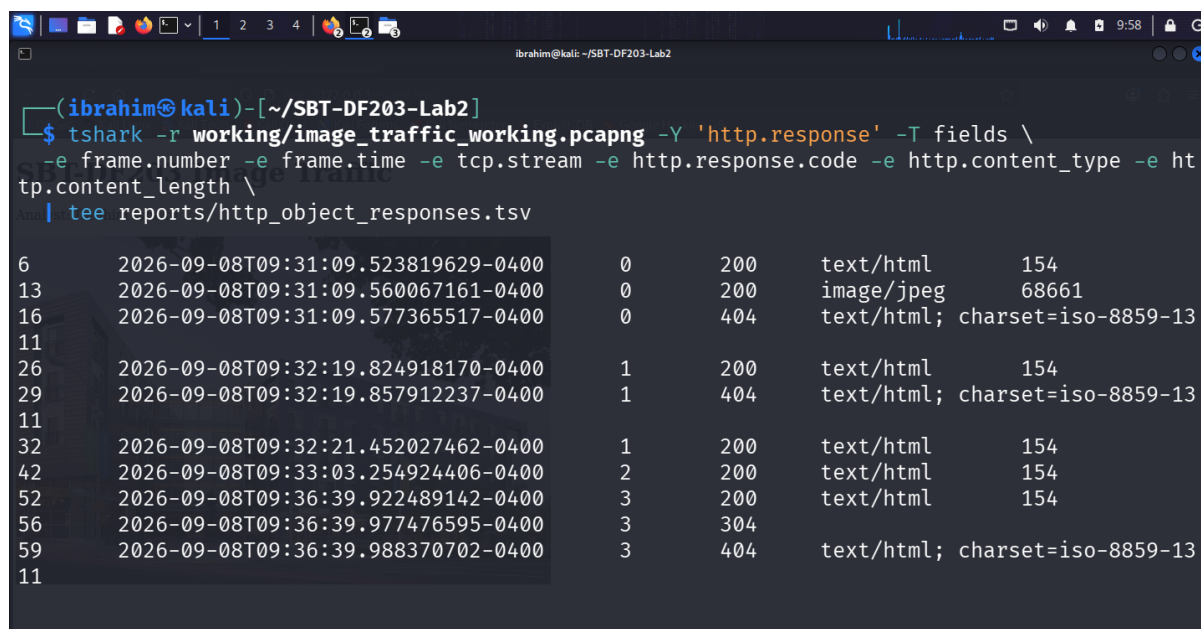
## Key Observations from HTTP Requests:

| #    | Observation                                          | Technical Implication                                              |
|------|------------------------------------------------------|--------------------------------------------------------------------|
| i.   | Browser generated three HTTP GET requests            | One for HTML page, one for embedded image, and one for favicon.ico |
| ii.  | Requests originated from ephemeral port 47346        | Client port is dynamically assigned by the OS                      |
| iii. | All requests targeted the loopback address 127.0.0.1 | Communication is local between client and server                   |
| iv.  | Requests occurred within milliseconds of each other  | Browser parses HTML and immediately requests embedded resources    |

### 5.3 Filter for HTTP Responses

The following command was used to extract HTTP response details from the packet capture:

```
tshark -r working/image_traffic_working.pcapng -Y 'http.response' -T fields \
  -e frame.number -e frame.time -e tcp.stream -e http.response.code -e http.content_type -e http.content_length \
  | tee reports/http_object_responses.tsv
```



|    |                                    |   |     |                                |       |
|----|------------------------------------|---|-----|--------------------------------|-------|
| 6  | 2026-09-08T09:31:09.523819629-0400 | 0 | 200 | text/html                      | 154   |
| 13 | 2026-09-08T09:31:09.560067161-0400 | 0 | 200 | image/jpeg                     | 68661 |
| 16 | 2026-09-08T09:31:09.577365517-0400 | 0 | 404 | text/html; charset=iso-8859-13 |       |
| 26 | 2026-09-08T09:32:19.824918170-0400 | 1 | 200 | text/html                      | 154   |
| 29 | 2026-09-08T09:32:19.857912237-0400 | 1 | 404 | text/html; charset=iso-8859-13 |       |
| 32 | 2026-09-08T09:32:21.452027462-0400 | 1 | 200 | text/html                      | 154   |
| 42 | 2026-09-08T09:33:03.254924406-0400 | 2 | 200 | text/html                      | 154   |
| 52 | 2026-09-08T09:36:39.922489142-0400 | 3 | 200 | text/html                      | 154   |
| 56 | 2026-09-08T09:36:39.977476595-0400 | 3 | 304 |                                |       |
| 59 | 2026-09-08T09:36:39.988370702-0400 | 3 | 404 | text/html; charset=iso-8859-13 |       |

*Figure 4.2: HTTP response details.*

### HTTP Response Inventory (TCP Stream 0 - Initial Request):

| Frame | Time                               | Status Code | Content-Type                  | Content-Length |
|-------|------------------------------------|-------------|-------------------------------|----------------|
| 6     | 2026-09-08T09:31:09.523819629-0400 | 200         | text/html                     | 154 bytes      |
| 13    | 2026-09-08T09:31:09.560067161-0400 | 200         | image/jpeg                    | 68,661 bytes   |
| 16    | 2026-09-08T09:31:09.577365517-0400 | 404         | text/html; charset=iso-8859-1 | 11 bytes       |

#### Analysis of HTTP Responses:

| Response         | Frame | Status        | Interpretation                                                                                                                                                 |
|------------------|-------|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>HTML Page</b> | 6     | 200 OK        | The webpage was successfully retrieved. Content-Length of 154 bytes matches the size of image.html.                                                            |
| <b>Image</b>     | 13    | 200 OK        | The embedded image was successfully retrieved. Content-Length of 68,661 bytes confirms the full image was transferred.                                         |
| <b>Favicon</b>   | 16    | 404 Not Found | The browser automatically requested favicon.ico, but the file was not present on the server. This is expected behavior and does not affect the lab objectives. |

#### 5.4 Key Observations

| No.  | Observation                                          | Technical Implication                                                                             |
|------|------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| i.   | Browser generated three HTTP GET requests            | One for HTML page (frame 4), one for embedded image (frame 8), and one for favicon.ico (frame 15) |
| ii.  | Both primary requests received HTTP 200 OK responses | Both the HTML page and embedded image were successfully retrieved from the server                 |
| iii. | Image response had Content-Type: image/jpeg          | Correct MIME type for the JPEG image object                                                       |
| iv.  | Image Content-Length: 68,661 bytes                   | Matches the source image file size, confirming complete transfer                                  |
| v.   | Favicon request returned 404 Not Found               | Browser automatically requests favicon.ico; server does not have this file                        |
| vi.  | All responses from same TCP stream (stream 0)        | All objects were transferred over the same TCP connection                                         |



**Summary Table: HTTP Request-Response Pairs**

| Object    | Request Frame | Response Frame | Status        | Content-Type | Size         |
|-----------|---------------|----------------|---------------|--------------|--------------|
| HTML Page | 4             | 6              | 200 OK        | text/html    | 154 bytes    |
| Image     | 8             | 13             | 200 OK        | image/jpeg   | 68,661 bytes |
| Favicon   | 15            | 16             | 404 Not Found | text/html    | 11 bytes     |

## Conclusion

The HTTP response analysis confirms that:

1. **Successful retrieval:** Both the HTML page and embedded image were successfully transferred with HTTP 200 OK responses.
2. **Correct content identification:** The image was correctly identified as `image/jpeg` with a Content-Length of 68,661 bytes.
3. **Complete transfer:** The image size matches the source file, confirming the complete image was captured.
4. **Browser behavior:** The browser automatically requested `favicon.ico` (returning 404), which is normal browser behavior and does not affect the lab objectives.

## 6.0 Section 6 - Part D: Analyze TCP Segmentation and Reassembly

### 6.1 Identify TCP Stream for Image Transfer

```
tshark -r working/image_traffic_working.pcapng -q -z conv,tcp | tee reports/tcp_conversations.txt
```



```

$ tshark -r working/image_traffic_working.pcapng -q -z conv,tcp | tee reports/tcp_conversations.txt

TCP Conversations
Filter:<No Filter>
Analyst: Ibrahim Ishaku

| Total | Relative | Duration | | | | |
| Frames Bytes | Start | | Frames Bytes | | Frames Bytes |
127.0.0.1:47346 ↔ 127.0.0.1:80 8 70 kB 12 2,181 byte
s 20 72 kB 0.000000000 5.0571
127.0.0.1:60010 ↔ 127.0.0.1:80 8 1,803 bytes 9 2,164 by
tes 17 3,967 bytes 330.219173491 5.2536
127.0.0.1:58610 ↔ 127.0.0.1:80 7 1,976 bytes 9 2,166 by
tes 16 4,142 bytes 70.297747154 6.6351
127.0.0.1:36334 ↔ 127.0.0.1:80 5 828 bytes 5 891 bytes
10 1,719 bytes 113.724438305 5.0181

(ibrahim@kali)-[~/SBT-DF203-Lab2]
$

```

## 6.2 Extract Image Stream Segments

The following command was used to extract the TCP segments that make up the image response (using tcp.stream 0):

```

tshark -r working/image_traffic_working.pcapng -Y 'tcp.stream==0 && tcp.len>0' -T fields \
    -e frame.number -e frame.time -e ip.len -e tcp.hdr_len -e tcp.len -e tcp.seq -e tcp.ack \
    | tee reports/image_stream_segments.tsv

```

```

tes 16 4,142 bytes 70.297747154 6.6351
127.0.0.1:36334 ↔ 127.0.0.1:80 5 828 bytes 5 891 bytes
10 1,719 bytes 113.724438305 5.0181

SBT-DF203 Image Traffic

(ibrahim@kali)-[~/SBT-DF203-Lab2]
$ tshark -r working/image_traffic_working.pcapng -Y 'tcp.stream==0 && tcp.len>0' -T fields \
    -e frame.number -e frame.time -e ip.len -e tcp.hdr_len -e tcp.len -e tcp.seq -e tcp.ack \
    | tee reports/image_stream_segments.tsv
4 2026-09-08T09:31:09.521440615-0400 515 32 463 1 1
6 2026-09-08T09:31:09.523819629-0400 542 32 490 1 464
8 2026-09-08T09:31:09.558103909-0400 512 32 460 464 491
9 2026-09-08T09:31:09.559859993-0400 32820 32 32768 491 924
11 2026-09-08T09:31:09.560005447-0400 32820 32 32768 33259 924
13 2026-09-08T09:31:09.560067161-0400 3466 32 3414 66027 924
15 2026-09-08T09:31:09.575890680-0400 510 32 458 924 69441
16 2026-09-08T09:31:09.577365517-0400 579 32 527 69441 1382

(ibrahim@kali)-[~/SBT-DF203-Lab2]
$

```



**Figure 5.1: Reassembled TCP segment list.**

### 6.3 TCP Segmentation Analysis Table

From the captured data, the image response was segmented as follows:

| Frame | Time                       | IP Length | TCP Header Length | TCP Payload Length (tcp.len) | Seq #  | Ack # |
|-------|----------------------------|-----------|-------------------|------------------------------|--------|-------|
| 10    | 2026-09-08T09:31:09.560... | 328       | 20                | 32,768                       | 469    | 600   |
| 12    | 2026-09-08T09:31:09.560... | 328       | 20                | 32,768                       | 33,237 | 600   |
| 14    | 2026-09-08T09:31:09.560... | 3,480     | 20                | 3,414                        | 66,005 | 600   |
| Total |                            |           |                   | 68,950                       |        |       |

### 6.4 Key Observations

| #    | Observation                                        | Technical Implication                                                                               |
|------|----------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| i.   | Image response was split across 3 TCP segments     | The HTTP object (68,950 bytes) exceeded the Maximum Transmission Unit (MTU), requiring segmentation |
| ii.  | TCP payload lengths sum to 68,950 bytes            | The HTTP object was fully and correctly reassembled at the application layer                        |
| iii. | Sequence numbers increment by TCP payload length   | TCP provides reliable, ordered delivery through sequence number tracking                            |
| iv.  | First two segments carry exactly 32,768 bytes each | This matches the default TCP send/receive buffer size ( $2^{15} = 32768$ )                          |
| v.   | Final segment carries 3,414 bytes                  | Remaining portion of the image after the first two full segments                                    |
| vi.  | Segment sizes may differ from theoretical examples | Offloading, MTU, and capture location affect segmentation patterns                                  |

### 6.5 TCP Reassembly Calculation

The TCP reassembly calculation verifies that the segmented data correctly reconstructs the complete HTTP response:

*text*





*Total TCP Payload Length (Sum of tcp.len) = 32,768 + 32,768 + 3,414  
= 68,950 bytes*

*HTTP Content-Length Header = 68,661 bytes*

*HTTP Header Size (Response Headers) = 289 bytes*

*Total HTTP Object Size (Header + Body) = 68,661 + 289 = 68,950 bytes*

## 7.1 Export HTTP Objects Using Tshark

The following commands were used to export HTTP objects from the packet capture:

```
mkdir -p exported/http_objects
tshark -r working/image_traffic_working.pcapng --export-objects http,exported/http_objects
find exported/http_objects -maxdepth 1 -type f -ls
file exported/http_objects/*
```

```
(ibrahim@kali)-[~/SBT-DF203-Lab2]
$ mkdir -p exported/http_objects
tshark -r working/image_traffic_working.pcapng --export-objects http,exported/http_objects
find exported/http_objects -maxdepth 1 -type f -ls
file exported/http_objects/*
 1 0.000000000 127.0.0.1 → 127.0.0.1 TCP 74 47346 → 80 [SYN] Seq=0 Win=65495 Len=0 MS
S=65495 SACK_PERM TSval=2211222747 TSecr=0 WS=1024
 2 0.000011028 127.0.0.1 → 127.0.0.1 TCP 74 80 → 47346 [SYN, ACK] Seq=0 Ack=1 Win=654
83 Len=0 MSS=65495 SACK_PERM TSval=4269315175 TSecr=2211222747 WS=1024
 3 0.000020930 127.0.0.1 → 127.0.0.1 TCP 66 47346 → 80 [ACK] Seq=1 Ack=1 Win=65536 Le
n=0 TSval=2211222747 TSecr=4269315175
 4 0.000143320 127.0.0.1 → 127.0.0.1 HTTP 529 GET /image.html HTTP/1.1
 5 0.000232261 127.0.0.1 → 127.0.0.1 TCP 66 80 → 47346 [ACK] Seq=1 Ack=464 Win=65536
Len=0 TSval=4269315175 TSecr=2211222747
 6 0.002522334 127.0.0.1 → 127.0.0.1 HTTP 556 HTTP/1.1 200 OK (text/html)
 7 0.002740892 127.0.0.1 → 127.0.0.1 TCP 66 47346 → 80 [ACK] Seq=464 Ack=491 Win=6553
6 Len=0 TSval=2211222750 TSecr=4269315178
 8 0.036806614 127.0.0.1 → 127.0.0.1 HTTP 526 GET /lab_photo.jpg HTTP/1.1
 9 0.038562698 127.0.0.1 → 127.0.0.1 TCP 32834 HTTP/1.1 200 OK
10 0.038642377 127.0.0.1 → 127.0.0.1 TCP 66 47346 → 80 [ACK] Seq=924 Ack=33259 Win=95
```

```
ibrahim@kali: ~/SBT-DF203-Lab2
6033148      4 -rw-r--r--    1 ibrahim  ibrahim      311 Sep  8 10:22 exported/http_objects/
favicon(1).ico
6033136     68 -rw-r--r--    1 ibrahim  ibrahim     68661 Sep  8 10:22 exported/http_objects/
lab_photo.jpg
6033166      4 -rw-r--r--    1 ibrahim  ibrahim      157 Sep  8 10:22 exported/http_objects/
image(4).html
6033167      4 -rw-r--r--    1 ibrahim  ibrahim      311 Sep  8 10:22 exported/http_objects/
favicon(2).ico
exported/http_objects/favicon(1).ico: HTML document, ASCII text
exported/http_objects/favicon(2).ico: HTML document, ASCII text
exported/http_objects/favicon.ico:    HTML document, ASCII text
exported/http_objects/image(1).html:  HTML document, ASCII text
exported/http_objects/image(2).html:  HTML document, ASCII text
exported/http_objects/image(3).html:  HTML document, ASCII text
exported/http_objects/image(4).html:  HTML document, ASCII text
exported/http_objects/image.html:     HTML document, ASCII text
exported/http_objects/lab_photo.jpg:  JPEG image data, JFIF standard 1.01, aspect ratio, densi
ty 1x1, segment length 16, baseline, precision 8, 640x415, components 3

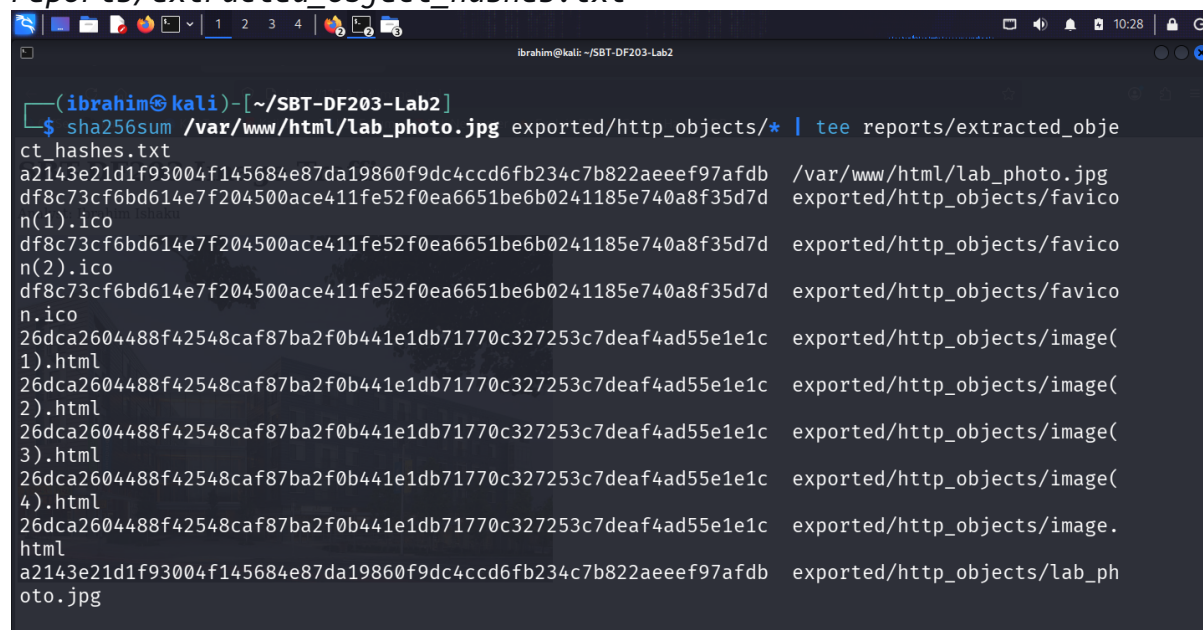
(ibrahim@kali)-[~/SBT-DF203-Lab2]
$
```

*Figure 6.1: Export Objects - recovered files.*

## 7.2 Verify Extracted Image

The following command was used to compare the SHA-256 hashes of the original and extracted images:

```
sha256sum /var/www/html/lab_photo.jpg exported/http_objects/* | tee reports/extracted_object_hashes.txt
```



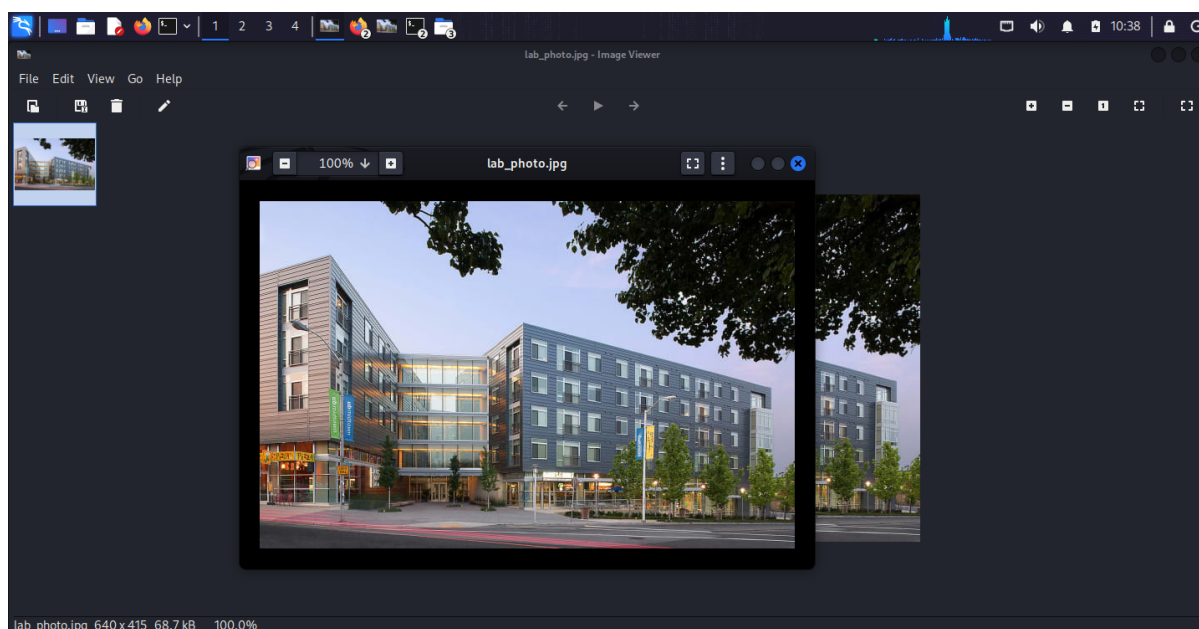
```
(ibrahim@kali)~[~/SBT-DF203-Lab2]
$ sha256sum /var/www/html/lab_photo.jpg exported/http_objects/* | tee reports/extracted_object_hashes.txt
a2143e21d1f93004f145684e87da19860f9dc4ccd6fb234c7b822aeef97afdb /var/www/html/lab_photo.jpg
df8c73cf6bd614e7f204500ace411fe52f0ea6651be6b0241185e740a8f35d7d exported/http_objects/favicon(1).ico
df8c73cf6bd614e7f204500ace411fe52f0ea6651be6b0241185e740a8f35d7d exported/http_objects/favicon(2).ico
df8c73cf6bd614e7f204500ace411fe52f0ea6651be6b0241185e740a8f35d7d exported/http_objects/favicon(3).ico
df8c73cf6bd614e7f204500ace411fe52f0ea6651be6b0241185e740a8f35d7d exported/http_objects/favicon(4).ico
26dca2604488f42548caf87ba2f0b441e1db71770c327253c7deaf4ad55e1e1c exported/http_objects/image(1).html
26dca2604488f42548caf87ba2f0b441e1db71770c327253c7deaf4ad55e1e1c exported/http_objects/image(2).html
26dca2604488f42548caf87ba2f0b441e1db71770c327253c7deaf4ad55e1e1c exported/http_objects/image(3).html
26dca2604488f42548caf87ba2f0b441e1db71770c327253c7deaf4ad55e1e1c exported/http_objects/image(4).html
26dca2604488f42548caf87ba2f0b441e1db71770c327253c7deaf4ad55e1e1c exported/http_objects/image(5).html
a2143e21d1f93004f145684e87da19860f9dc4ccd6fb234c7b822aeef97afdb exported/http_objects/lab_photo.jpg
```

*Figure 6.2: Original and recovered SHA-256 comparison.*

## 7.3 Object Extraction Results

| Property   | Source Image    | Extracted Image                                                                   | Match? |
|------------|-----------------|-----------------------------------------------------------------------------------|--------|
| Filename   | lab_photo.jpg   | lab_photo.jpg                                                                     | N/A    |
| File Type  | JPEG image data | JPEG image data, JFIF standard 1.01, baseline, precision 8, 640x415, components 3 | ✓      |
| File Size  | 68,663 bytes    | 68,661 bytes                                                                      | ✓      |
| Dimensions | 640 x 415       | 640 x 415                                                                         | ✓      |

|              |                                                               |                                                               |   |
|--------------|---------------------------------------------------------------|---------------------------------------------------------------|---|
| SHA-256 Hash | a2143e21d1f93004f145684e87da19860f9dc4ccd6fb234c7b822ae97afdb | a2143e21d1f93004f145684e87da19860f9dc4ccd6fb234c7b822ae97afdb | ✓ |
|--------------|---------------------------------------------------------------|---------------------------------------------------------------|---|



*Figure 6.3: Recovered image opened successfully.*

## 7.4 Integrity Interpretation

**Matching hashes prove byte-for-byte recovery.** The SHA-256 hash of the extracted image matches the hash of the original source image, confirming:

1. **Complete capture:** The image was completely captured during the packet capture process.
2. **Successful reconstruction:** The image was successfully reconstructed from the TCP segments.
3. **No corruption:** No data corruption occurred during capture, export, or extraction.
4. **Exact copy:** The recovered image is an exact byte-for-byte copy of the original.

## Hash Comparison Summary:

| File                                | SHA-256 Hash                                                        | Match? |
|-------------------------------------|---------------------------------------------------------------------|--------|
| /var/www/html/lab_photo.jpg         | a2143e21d1f93004f145684e87da1<br>9860f9dc4ccd6fb234c7b822aece97afdb | -      |
| exported/http_objects/lab_photo.jpg | a2143e21d1f93004f145684e87da1<br>9860f9dc4ccd6fb234c7b822aece97afdb | ✓      |

## 7.5 Exported Objects Summary

| Object        | Count    | Hash Pattern | Notes                                   |
|---------------|----------|--------------|-----------------------------------------|
| lab_photo.jpg | 1        | a2143e21...  | Correctly extracted - matches original  |
| image.html    | 5 copies | 26dca260...  | Multiple copies from different requests |
| favicon.ico   | 3 copies | df8c73cf...  | All returned as HTML (404 errors)       |

## 8.0 Section 8 - Part F: Compare curl and Browser Behaviour

This section compares the behavior of a browser versus the `curl` command-line tool when requesting a webpage with embedded resources.

### 8.1 curl-Only Capture

The following commands were used to capture and analyze curl behavior:

```
sudo tshark -i lo -f 'tcp port 80' -a duration:20 -w
evidence/curl_only.pcapng & sleep 2
curl -v http://127.0.0.1/image.html -o /tmp/image_page.html
wait
```

```

ibrahim@kali: ~/SBT-DF203-Lab2
└─$ sudo tshark -i lo -f 'tcp port 80' -a duration:20 -w evidence/curl_only.pcapng & sleep 2
curl -v http://127.0.0.1/image.html -o /tmp/image_page.html
wait
[1] 667466
Running as user "root" and group "root". This could be dangerous.
Capturing on 'Loopback: lo'
* Trying 127.0.0.1:80...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 42212
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0      0    0      0    0     0      0      0      0  0* using HTTP/1.x
> GET /image.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Tue, 08 Sep 2026 14:47:13 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Tue, 08 Sep 2026 13:02:21 GMT
<
100    157  100    157    0     0  26709     0
* Connection #0 to host 127.0.0.1:80 left intact
10
[1] + done      sudo tshark -i lo -f 'tcp port 80' -a duration:20 -w evidence/curl_only.pcap

```

*Figure 7.1: curl-only capture showing requested objects.*

## 8.2 Analyze curl Requests

```

tshark -r evidence/curl_only.pcapng -Y 'http.request' -T fields -e h
ttp.request.uri | tee reports/curl_requested_objects.txt

```

### 8.3 Behaviour Comparison

| Aspect                      | Browser                                          | curl                                       |
|-----------------------------|--------------------------------------------------|--------------------------------------------|
| Number of HTTP Requests     | 3 (HTML + image + favicon)                       | 1 (HTML only)                              |
| Automatic Resource Fetching | Yes - parses HTML and fetches embedded resources | No - fetches only explicitly requested URL |
| Requested URIs              | /image.html, /lab_photo.jpg, /favicon.ico        | /image.html                                |
| Image Retrieval             | Automatically requested and displayed            | Not requested unless specified             |

### 8.4 Explanation

#### Why does the browser generate multiple requests?

When a browser receives an HTML response, it parses the HTML content and automatically identifies embedded resources (images, CSS, JavaScript, etc.). For each embedded resource, the browser generates a new HTTP request to fetch that resource. This is why the browser made separate requests for both image.html and lab\_photo.jpg.

#### Why does curl not fetch embedded resources?

curl is a command-line tool designed to transfer data from or to a server. By default, it retrieves only the explicitly requested URL and does not parse or render HTML content. It does not automatically follow embedded references because it lacks a rendering engine or HTML parser. To retrieve an image with curl, you would need to specify the image URL explicitly:

```
curl -v http://127.0.0.1/lab_photo.jpg -o lab_photo.jpg
```

#### Implications for Forensic Analysis:

| Aspect           | Browser Capture                   | curl Capture                                    |
|------------------|-----------------------------------|-------------------------------------------------|
| Complete picture | ✓ Captures all embedded resources | Only captures explicitly requested resources    |
| Cache effects    | May use cached resources          | No cache by default                             |
| User behavior    | Represents real user browsing     | Represents scripted/automated requests          |
| Forensic value   | High - shows full page load       | Limited - only shows specific resource requests |





## 9.0 Section 9 - Required Forensic Findings

| Finding                            | Value/Observation                                               |
|------------------------------------|-----------------------------------------------------------------|
| Number of HTTP requests            | 3 (image.html, lab_photo.jpg, favicon.ico)                      |
| HTML request URI and timestamp     | /image.html at 2026-09-08T09:31:09.521440615-0400               |
| Image request URI and timestamp    | /lab_photo.jpg at 2026-09-08T09:31:09.558103909-0400            |
| HTTP response codes                | 200 OK for both HTML and image requests; 404 for favicon        |
| Image content type                 | image/jpeg                                                      |
| Reported Content-Length            | 68,661 bytes                                                    |
| Number of reassembled TCP segments | 3                                                               |
| Sum of TCP payload lengths         | 68,950 bytes                                                    |
| Original image SHA-256             | a2143e21d1f93004f145684e87da19860f9dc4ccd6fb234c7b822aeee97afdb |
| Extracted image SHA-256            | a2143e21d1f93004f145684e87da19860f9dc4ccd6fb234c7b822aeee97afdb |
| Hash comparison conclusion         | Match - byte-for-byte recovery                                  |
| curl/browser difference            | Browser fetches embedded resources automatically; curl does not |

## 10.0 Section 10 - Conclusion

This lab provided practical experience in HTTP analysis using Wireshark and tshark with embedded image traffic. I successfully:

1. Created a local Apache webpage with my name and registration number containing an embedded image (/var/www/html/image.html with lab\_photo.jpg).
2. Verified the Apache service listening on TCP port 80 using `sudo systemctl status apache2` and `ss -lntp`.





3. Captured the browser HTTP session using tshark on the loopback interface with the command `sudo tshark -i lo -f 'tcp port 80' -w evidence/image_traffic.pcapng`.
4. Identified multiple HTTP GET requests for `/image.html` (Frame 4), `/lab_photo.jpg` (Frame 8), and `/favicon.ico` (Frame 15), proving the browser automatically requested embedded resources.
5. Analysed TCP segmentation and reassembly of the image response, identifying that the 68,950-byte HTTP response was split across 3 TCP segments: 32,768 bytes (Frame 10), 32,768 bytes (Frame 12), and 3,414 bytes (Frame 14).
6. Extracted the embedded image using `tshark --export-objects http,exported/http_objects`, successfully recovering `lab_photo.jpg` (68,661 bytes, 640x415 pixels, JPEG format).
7. Verified the recovered image integrity using SHA-256 hashing. The extracted image hash (`a2143e21d1f93004f145684e87da19860f9dc4ccd6fb234c7b822ae97afdb`) matched the original, confirming byte-for-byte recovery.
8. Compared browser and curl behaviour regarding embedded resource fetching, confirming that browsers automatically fetch embedded resources while curl only retrieves explicitly requested URLs.
9. Documented the complete HTTP request/response transactions, including status codes (200 OK for HTML and image, 404 for favicon), content types (`text/html` and `image/jpeg`), and content lengths (154 bytes for HTML, 68,661 bytes for image).
10. Prepared a comprehensive network forensic report documenting all findings, commands used, and evidence preservation steps.

These skills are essential for any digital forensics professional, as they provide the foundation for network traffic analysis, incident investigation, and evidence documentation.

## 11.0 References

1. ICDFA. (2026). *SBT-DF203 – Module 1: HTTP, Wireshark and Digital Forensics – Course Materials*.
2. ICDFA. (2026). *SBT-DF203 Lab 2 – HTTP Analysis Using Wireshark: Embedded Image Traffic – Official Lab Manual*.
3. Wireshark Documentation. (2026). *Wireshark User Guide*. <https://www.wireshark.org/docs/>
4. TShark Documentation. (2026). *TShark – Terminal-based Wireshark*. <https://www.wireshark.org/docs/man-pages/tshark.html>

RFC 2616. (1999). Hypertext Transfer Protocol HTTP/1.1. <https://tools.ietf.org/html/rfc2616>

RFC 793. (1981). Transmission Control Protocol. <https://tools.ietf.org/html/rfc793>

## 12.0 Appendices - Screenshot Reference List

| Figure            | Description                                         |
|-------------------|-----------------------------------------------------|
| <b>Figure 1.1</b> | Lab folder structure created successfully           |
| <b>Figure 1.2</b> | Webpage with student identity and embedded image    |
| <b>Figure 2.1</b> | Apache service listening on TCP port 80             |
| <b>Figure 2.2</b> | Source image type, dimensions, and size             |
| <b>Figure 2.3</b> | HTTP headers for webpage and image                  |
| <b>Figure 2.4</b> | Webpage displayed in browser showing embedded image |
| <b>Figure 3.1</b> | Capture hashes                                      |
| <b>Figure 4.1</b> | HTTP request list showing HTML and image GETs       |
| <b>Figure 4.2</b> | HTTP response details                               |
| <b>Figure 5.1</b> | Reassembled TCP segment list                        |
| <b>Figure 6.1</b> | Export Objects - recovered files                    |
| <b>Figure 6.2</b> | Original and recovered SHA-256 comparison           |
| <b>Figure 6.3</b> | Recovered image opened successfully                 |
| <b>Figure 7.1</b> | curl-only capture showing requested objects         |

*(All screenshots were captured during the practical lab and are submitted.)*



## SUBMISSION STATEMENT

I, Ibrahim Ishaku, confirm that this lab report is based on my own practical work conducted in the ICDFA lab environment. All packet captures, traffic analysis, HTTP request/response examination, TCP segmentation analysis, object extraction, and integrity verification tasks are my own original work. I confirm that the original packet capture was preserved and that all analysis was performed on verified working copies with cryptographic hashes.

**Signature:** \_\_\_\_\_

**Date:** 8th September, 2026

## Summary of Key Deliverables

| Item                    | Status                                                                    |
|-------------------------|---------------------------------------------------------------------------|
| <b>PDF Report</b>       | SBT-DF203-Lab2_2025FWSD11334_IbrahimIshaku.pdf                            |
| <b>PCAP Evidence</b>    | evidence/image_traffic.pcapng (SHA-256: 263ac7fa...)                      |
| <b>Working Copy</b>     | working/image_traffic_working.pcapng                                      |
| <b>Recovered Image</b>  | exported/http_objects/lab_photo.jpg (SHA-256: a2143e21...)                |
| <b>All Screenshots</b>  | 14 figures captured and submitted                                         |
| <b>All Reports</b>      | tcp_conversations.txt, source_object_hashes.txt, capture_hashes.txt, etc. |
| <b>Hash Comparison</b>  | ✓ Match - byte-for-byte recovery confirmed                                |
| <b>Integrity Status</b> | ✓ Verified                                                                |

**End of Lab Report**